ELTE EÖTVÖS LORÁND UNIVERSITY

Algorithms and Data Structures I.

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

2026. február 23.

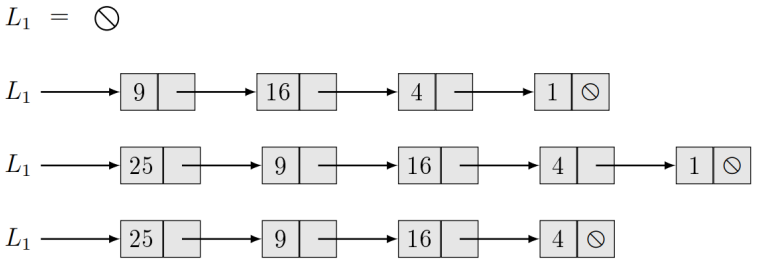
Content in this Practice I

Linked lists - One-way lists

Linked lists

- A **linked list** is a data structure in which the objects are arranged in a linear order. Unlike an array, however, in which the linear order is determined by the array indices, the order in a linked list is determined by a pointer in each object. *Source: Introduction to Algorithms by Cormen et. al.*
- Types:
  - Simple one-way list (S1L)
  - One-way lists with header node (H1L)
  - Cyclic one-way list (C1L)
  - Simple two-way or doubly linked list (S2L)
  - Cyclic two-way list (C2L) with header

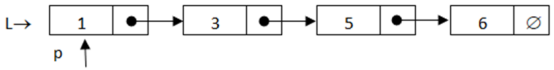
Simple one-way list (S1L)



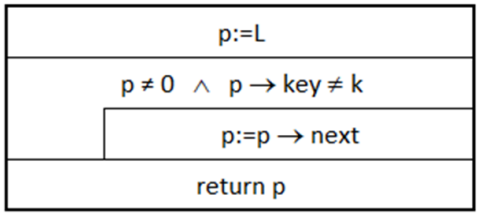
- Each list element has its general data, and a pointer pointing to the next element in the list. If it's the last element, the pointer is NULL.
- L points to the first element, but L is NOT an element in itself, nor is it a type of E1.

E1
$+key : \mathcal{T}$ ... // satellite data may come here $+next$: **E1***
$+$**E1**() { $next := \oslash$ }

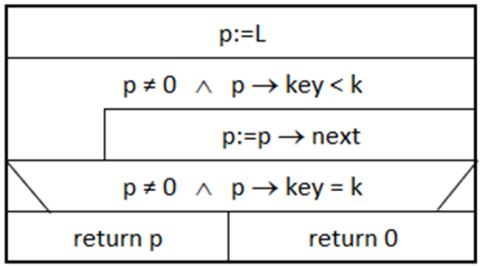
Simple one-way list (S1L) - Iterating and Searching



S1L_search(L:E1*, k :T) : E1*

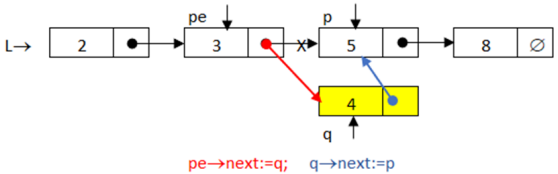


S1L_search(L:E1*, k :T) : E1*



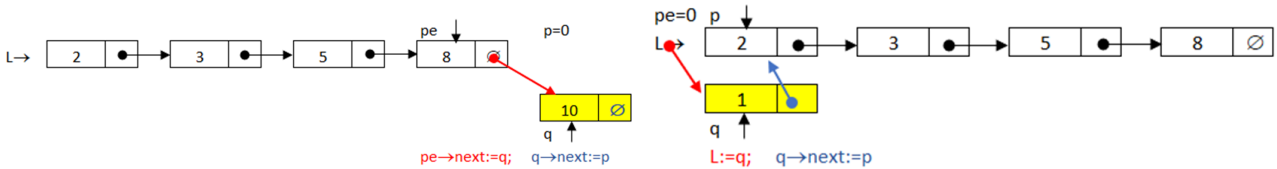
- Iterating:
  - p: pointer to one of the elements p: E1*
  - p→key: the value of the list element.
  - p→next: pointer value of the next element (if any).
  - Allocating a new element: p:= new E1
  - Deleting list element: delete p
    After deletion, referencing any fields is invalid.
- Searching:
  - Let L point to an ordered list of S1L. Find an element in L with key 'k'. If one such element exists, return with its pointer, otherwise return null.

Simple one-way list (S1L) - Insertion



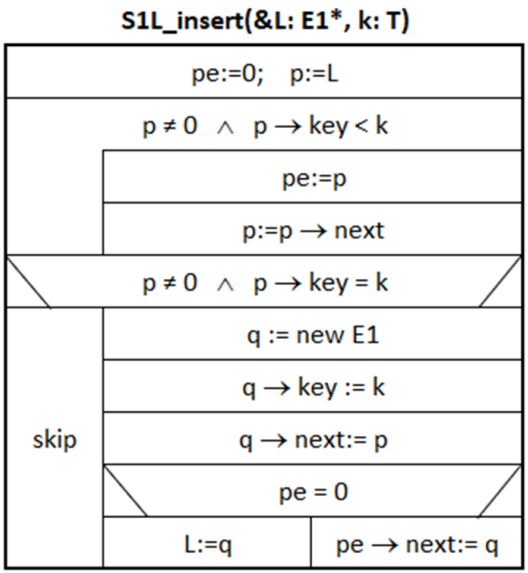
- Insertion:
  - Let L point to an ordered S1L. Insert a key into the list, and keep it ordered.
  - If such key already exists, do nothing. (set)
  - The position of the key can be in the four possible places:
  - The element should be somewhere in the middle,
  - The element is the largest, making it the last,
  - The element is the smallest, making it the first,
  - The list is empty.

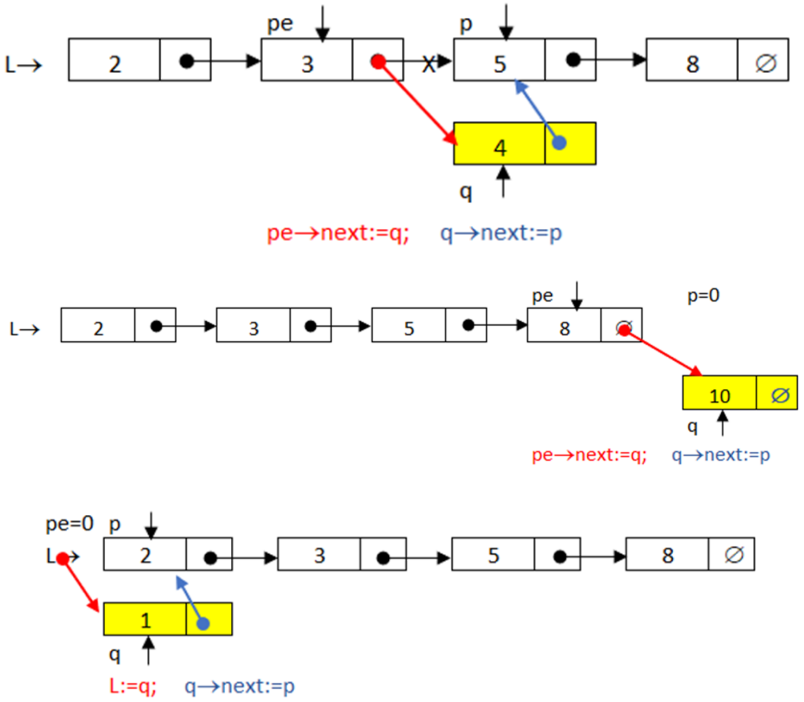
Simple one-way list (S1L) - Insertion



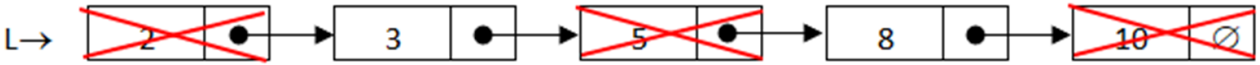
- At the end:
  - Inserting to the end of the list:
  - p is null since every element is smaller than the key to be inserted.
  - pe then is the last element, allowing us to insert after it.
- At the start:
  - Inserting at the beginning means that L should now point to the new element (pe is null):
  - Inserting into an empty list means that both p, and pe are null:

Simple one-way list (S1L) - Insertion



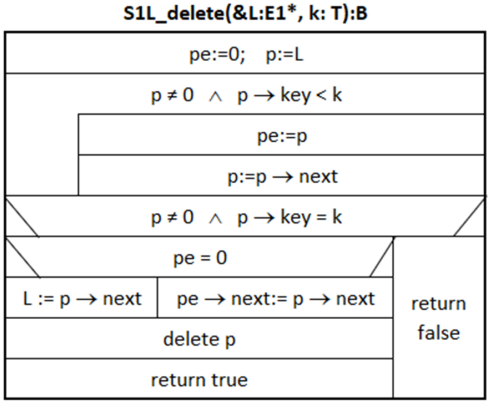


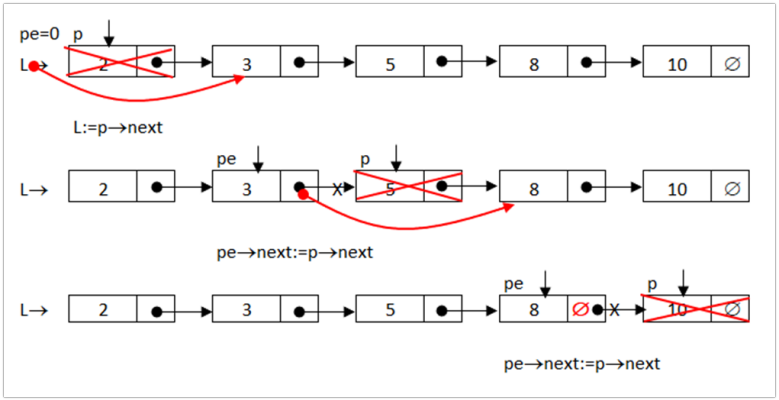
Simple one-way list (S1L) - Deletion



- Deletion:
  - Once again, L points to a sorted S1L. Delete an element if it exists.
  - If the deletion is successful (exists) return true, otherwise false.
  - To delete the element and keep the list intact, we need the previous element once again.
  - In case of (successful) deletion, the following cases are possible:
  - Deleting the first element.
  - Deleting an inner element,
  - Deleting the last element.

Simple one-way list (S1L) - Deletion

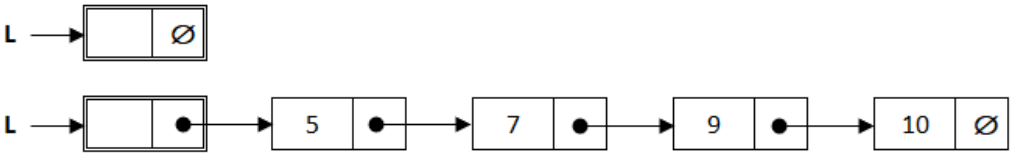




One-way lists with header node

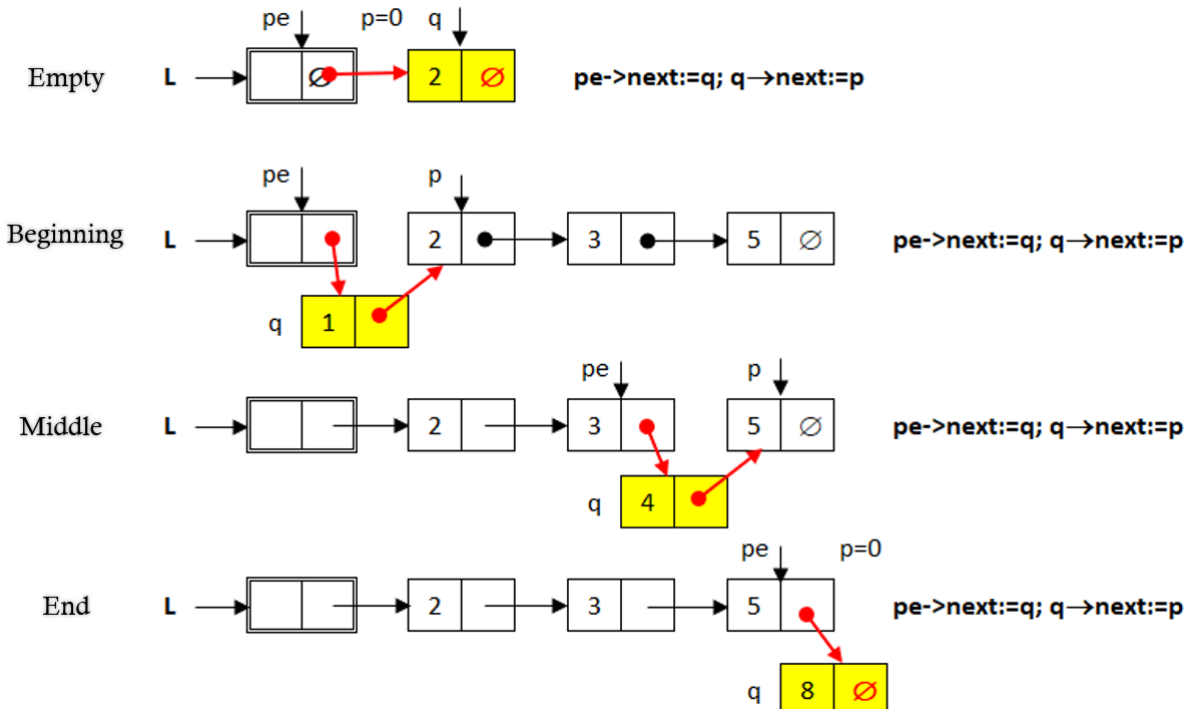
One-way lists with header node (H1L)

- Header element:
  - As we've seen these algorithms need to handle the beginning of the list differently from the other cases since pe is null.
  - To solve this issue, and avoid unnecessary branches we introduce a header element.
  - This way the list will always have an E1 element, sort of a placeholder!
  - This also means that we however don't store data in it.

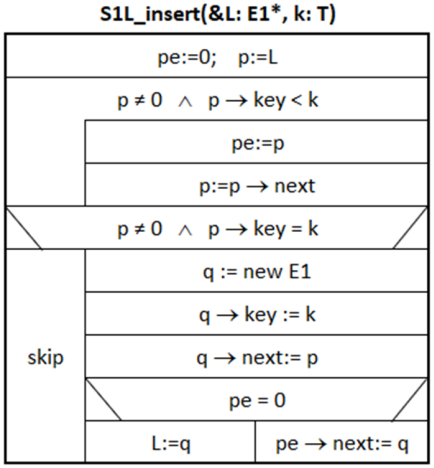


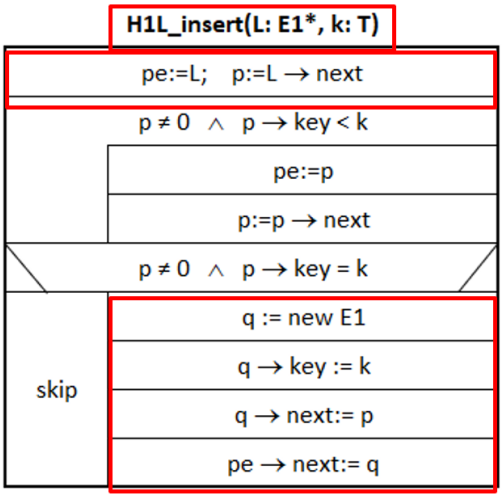
- Searching will be slightly different:
  - We start from the first element after the head

One-way lists with header node (H1L) - Insertion



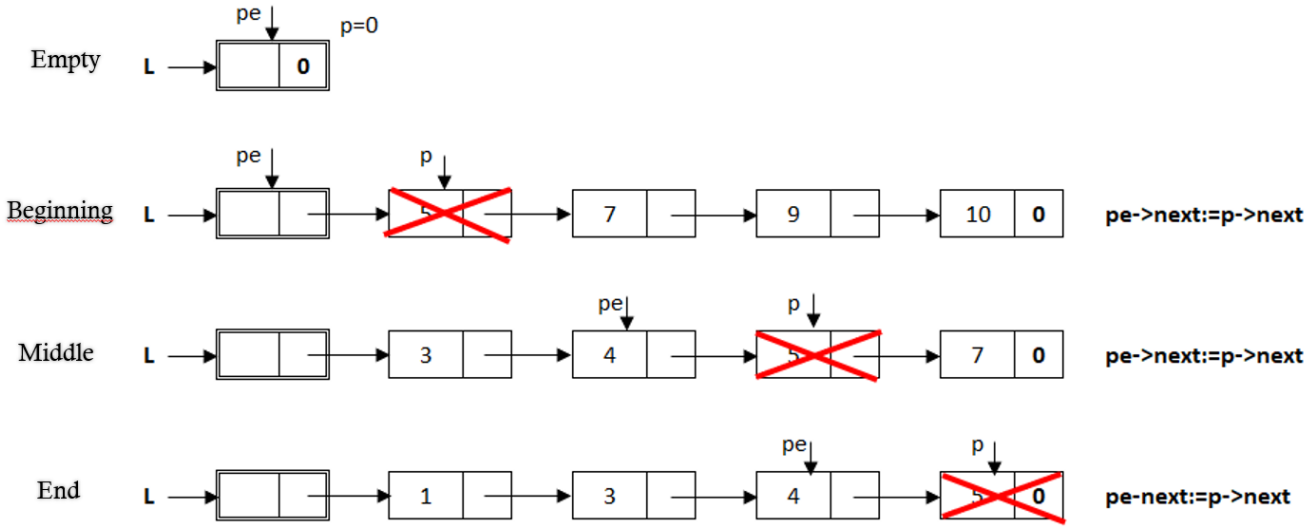
One-way lists with header node (H1L) - Insertion



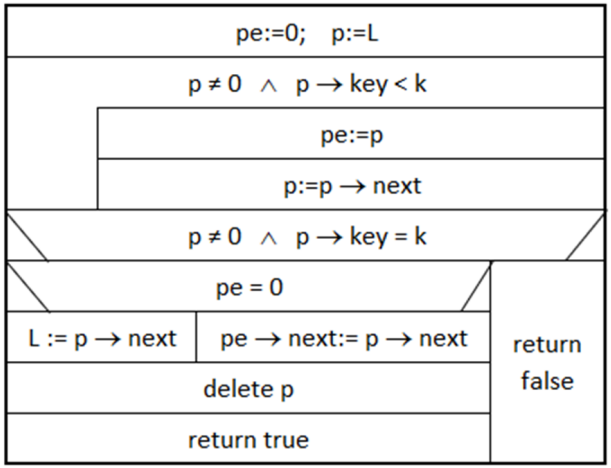


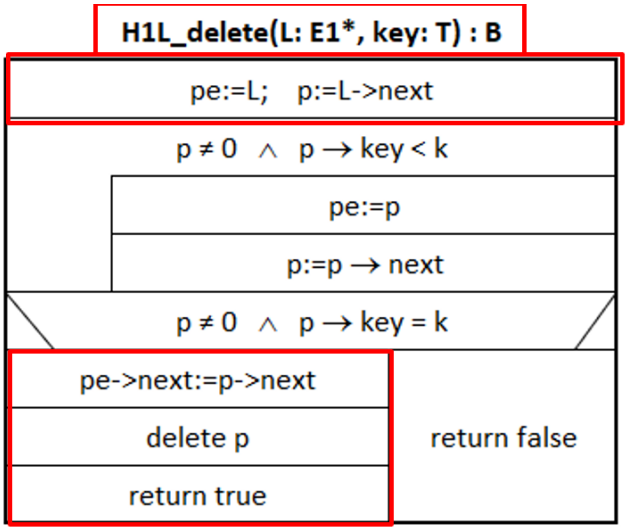
- Differences:
  - L is not passed by reference.
  - Start: pe is the header, p is the first element (can be null)
  - No branch is needed since pe is never null.

One-way lists with header node (H1L) - Deletion



One-way lists with header node (H1L) - Deletion





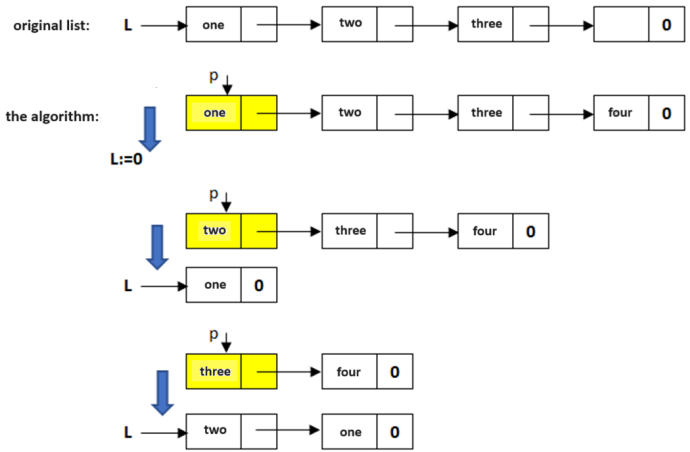
- Differences:
  - L is not passed by reference.
  - Start: pe is the header, p is the first element (can be null)
  - No branch is needed since pe is never null.

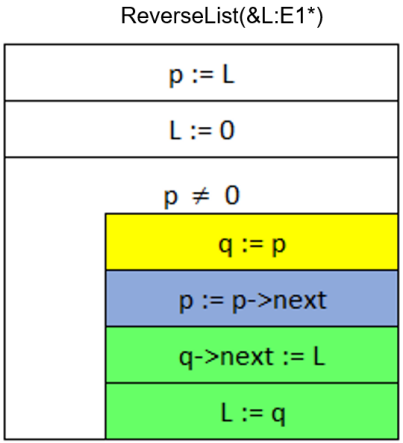
Illustration prob. for One-way l

Reversing an S1L list

- Let L point to an S1L. Reverse the order of elements.
- Input:
  - L $\rightarrow$ one $\rightarrow$ two $\rightarrow$ three $\rightarrow$ four
- Output:
  - L $\rightarrow$ four $\rightarrow$ three $\rightarrow$ two $\rightarrow$ one
- Idea:
  - 'Deconstruct' the original list, and p points to the first element.
  - Set L to empty list //L:=0
  - Iterating on the original list, and inserting each element right at the beginning of L will result in the reversed list.

Reversing an S1L list



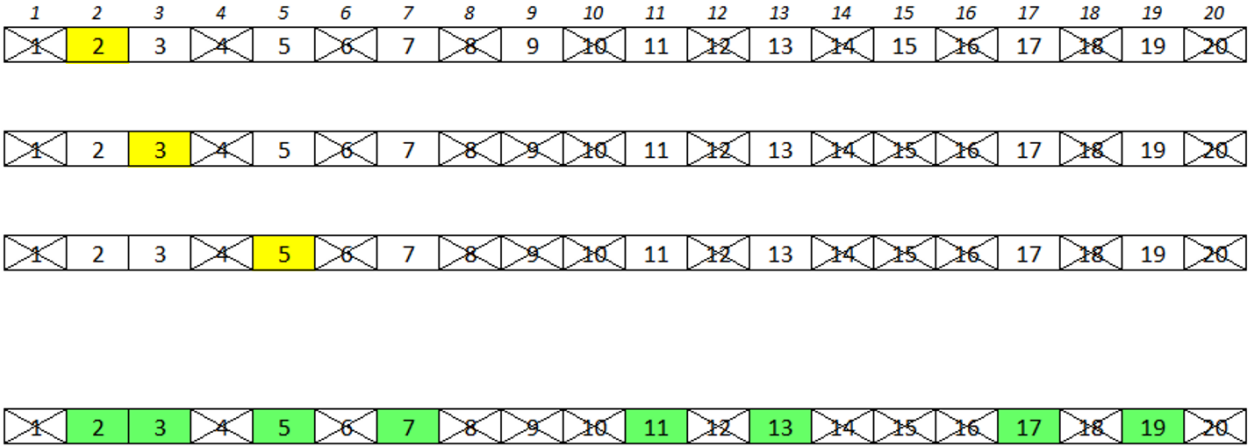


- q is the element to be inserted at the new beginning
- We must step to the next element in the previous list, otherwise, we'd lose track of the original list
- Inserting q: the previous first element should come after q, and q should be the new first element.

Sieve of Eratosthenes with S1L

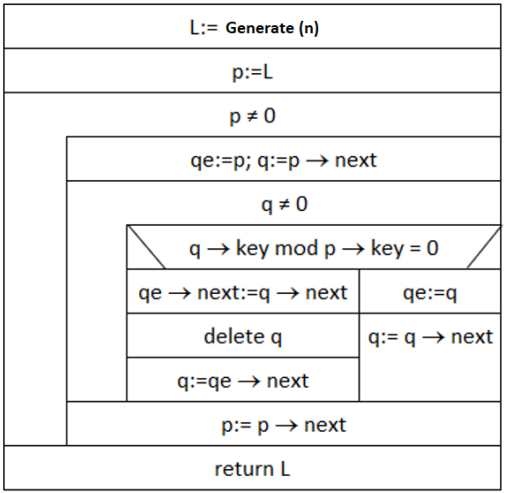
- Using Eratosthenes' algorithm, create a list of prime numbers smaller than n.
- Idea:
  - Create a list of integers smaller than n. 1 is not a prime, delete. 2 is the first prime, and from now on we have to delete every second element (since they are a multiple of 2, and therefore not primes.)
  - From the remaining elements, the next is 3, it's a prime, keep it, and delete the multiples of 3 once again.
  - And so on until we reach the last element. The remaining elements are all primes.

Sieve of Eratosthenes with S1L



Sieve of Eratosthenes with S1L

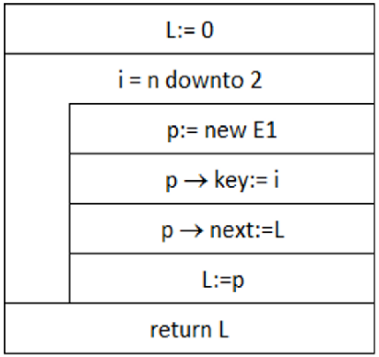
PrimeSieve(n:N):E1*



p points to next prime.

q will point to the follow-up elements, and we check if the given element is a multiple of p or not. If yes, delete.

Generate(n:N):E1*



Given that we deleted the multiples of p (and the multiples before it), the next element after p will also going to be a prime.

Ending

Thank you for your attention!